

ЛЕКЦИЯ 5 из 8

FishNet: архитектура, Observers и миграция проекта

Блок 2 — Продвинутый мультиплеер с FishNet • ~120 мин

КАРТА ЗАНЯТИЯ

- ① Почему FishNet — мотивация, бенчмарки, архитектура
- ② SyncVar и SyncType — замена NetworkVariable
- ③ RPC в FishNet — ObserversRpc, TargetRpc, маппинг с NGO
- ④ Observers — Area of Interest, условия видимости

Почему не только NGO?

①
Мотивация

✓ Что мы любим в NGO

Официальное решение Unity —
долгосрочная поддержка
Бесплатное, интеграция с Relay/Lobby
Отличный старт для обучения

⚠ Где NGO буксует

Нет встроенного Prediction
Observer-система примитивна
Нет Host Migration
Падение при 32+ клиентах и 500+
объектах

NGO — отличный фреймворк для прототипов и малых проектов. Для продакшена индустрия часто выбирает FishNet.

FishNet: кто и зачем

①
Мотивация



Архитектура

Декомпозированная:
ServerManager,
ClientManager,
TransportManager



Производительность

Меньше аллокаций, ниже
GC pressure, выше tick rate



Возможности

Встроенный Prediction,
мощные Observers, Host
Migration

- Open-source (MIT лицензия)
- Активное сообщество и документация
- Используется в коммерческих проектах

Бенчмарки: NGO vs FishNet

①

Мотивация

Метрика	NGO	FishNet	Разница
100 объектов, tick rate	30 Hz	30 Hz	≈ паритет
500 объектов, tick rate	20 Hz (просадка)	30 Hz (стабильно)	FishNet +50%
1000 объектов, GC alloc/frame	~2.4 KB	~0.3 KB	FishNet 8× меньше
Serialization overhead	Средний	Низкий	FishNet быстрее

При малом количестве объектов — разницы нет. При масштабировании — FishNet выигрывает кратно.

** Бенчмарки приблизительные, зависят от проекта. Всегда профилируйте свой конкретный случай.*

Архитектура FishNet: обзор компонентов

Архитекту

NetworkManager — Точка входа (НЕ синглтон!)

— **ServerManager** Серверная логика
— OnRemoteConnectionState

— **ClientManager** Клиентская логика

— **TransportManager**

— Tugboat (UDP)
— Bayou (WebSocket)
— Multipass

— **TimeManager** Тики, время

— **ObserverManager** АОI

— **PredictionManager** (Лекция 6)

Ключевое отличие

В отличие от NGO, FishNet
НЕ требует синглтона —
можно иметь несколько
NetworkManager

*Применение: split-screen,
тестирование*

Сравнение архитектур: NGO ↔ FishNet

Архитекту

NGO (Лекция 1)	FishNet
NetworkManager (синглтон)	NetworkManager (не синглтон)
UnityTransport (UDP)	TransportManager → Tugboat / Bayou
ConnectionManager	ServerManager + ClientManager
—	TimeManager (тики)
—	ObserverManager (AOI)

FishNet декомпозирует то, что NGO держит в одном компоненте → больше гибкости, но чуть больше настройки.

Transport: выбор канала связи

Архитекту



Tugboat

UDP-транспорт

По умолчанию

Минимальная задержка

≈ *UnityTransport*



Bayou

WebSocket-транспорт

Для WebGL-билдов

Работает через браузер

Аналога в NGO нет



Multipass

Несколько транспортов

Одновременно

Desktop + WebGL

Аналога в NGO нет

Выбор транспорта — конфигурация в Inspector. Код игры НЕ меняется. Это уровень абстракции, которого не хватает в NGO.

Жизненный цикл NetworkBehaviour в FishNet

Архитекту

Awake()

↓

OnStartNetwork() — зарег. в сети

↓

OnStartServer() только сервер

↓

OnStartClient() каждый клиент

↓

OnOwnershipClient(prev) новое!

↓ ... Runtime ...

OnStopClient()

↓

OnStopServer()

↓

OnStopNetwork()

Сравнение с NGO

NGO:

OnNetworkSpawn() — один метод для всех
OnNetworkDespawn()

FishNet:

OnStartServer() + OnStartClient() —
раздельно
OnStopServer() + OnStopClient()

Новое в FishNet:

OnOwnershipClient(prev) — реакция на
смену владельца

② **SyncVar** и **SyncType**

Замена `NetworkVariable` — меньше
бойлерплейта, больше возможностей

SyncVar: декларативная синхронизация

② SyncVar

NGO — Лекция 2

```
// NGO: NetworkVariable
public NetworkVariable<int> health =
    new(100,
        ReadPermission.Everyone,
        WritePermission.Server);

public override void OnNetworkSpawn() {
    health.OnValueChanged +=
        OnHealthChanged;
}
```

FishNet

```
// FishNet: SyncVar
[SyncVar(OnChange =
    nameof(OnHealthChanged))]
public int health = 100;

private void OnHealthChanged(
    int prev, int next,
    bool asServer) {
    UpdateHealthUI(next);
}
```

- ✓ **Меньше кода:** нет ручной подписки/отписки
- ✓ **Колбэк** объявляется атрибутом
- ✓ **Типы** — обычные C# поля ⚠ **По умолчанию** write доступен владельцу — нужно контролировать

SyncVar: кто читает, кто пишет

Параметр	Значение	Описание
ReadPermission.Observers	По умолчанию	Все наблюдатели видят значение
ReadPermission.OwnerOnly		Только владелец видит
ReadPermission.ExcludeOwner		Все кроме владельца
WritePermission.ServerOnly	По умолчанию	Только сервер меняет

```
[SyncVar(WritePermission = WritePermission.ServerOnly, ReadPermission = ReadPermission.Observers)]
public int score = 0;
```

SyncList: умнее, чем NetworkList

② SyncType

NGO — Лекции 2, 4

```
// NGO: NetworkList
NetworkList<InventorySlot> inventory;

// Один колбэк на всё
inventory.OnListChanged
+= OnInventoryChanged;
// switch по ChangeEvent.Type
```

FishNet

```
public readonly SyncList<InventorySlot>
inventory = new();

// Гранулярные колбэки
inventory.OnChange += OnChange;

void OnChange(SyncListOperation op,
int index, ...
bool asServer) { ... }
```

- ✓ **Операция + индекс + старое/новое значение в одном колбэке**
- ✓ **Дельта-синхронизация:** отправляется только изменение, не весь список

SyncDictionary и SyncHashSet — чего нет в NGO

② SyncType

SyncDictionary<K,V>

```
public readonly SyncDictionary
<string, int> playerScores = new();

// Добавление (только сервер)
playerScores["Alice"] = 150;
playerScores["Bob"] = 230;

playerScores.OnChange +=
(op, key, val, asServer)
=> UpdateScoreboard();
```

SyncHashSet<T>

```
public readonly SyncHashSet<ulong>
readyPlayers = new();

readyPlayers.Add(clientId);
readyPlayers.Remove(clientId);

bool allReady =
readyPlayers.Count ==
totalPlayers;
```

⚠ В NGO аналогов нет — приходилось эмулировать через NetworkVariable<custom struct>

③ RPC в FishNet

ObserversRpc, TargetRpc и маппинг с NGO

RPC: что изменилось

Направление	NGO (Лекции 1, 3)	FishNet
Клиент → Сервер	[ServerRpc]	[ServerRpc]
Сервер → Все клиенты	[ClientRpc]	[ObserversRpc] ← только наблюдателям!
Сервер → Один клиент	ClientRpcParams + TargetClientIds	[TargetRpc] + NetworkConnection

⚠ **Ключевое отличие:** [ObserversRpc] шлёт НЕ ВСЕМ клиентам, а только тем, кто НАБЛЮДАЕТ объект.
Если объект вне зоны AOI клиента — клиент НЕ получит RPC. Это экономит трафик.

ServerRpc в FishNet

③ RPC

NGO — Лекция 3

```
[ServerRpc(RequireOwnership = true)]
void ShootServerRpc(
    Vector3 direction,
    ServerRpcParams rpcParams
    = default)
{
    ulong sender =
        rpcParams.Receive
            .SenderClientId;
    if (sender != OwnerClientId)
        return;
}
```

FishNet

```
[ServerRpc(RequireOwnership = true)]
void ShootServerRpc(
    Vector3 direction)
{
    // RequireOwnership уже
    // проверяет владельца!
    // ...
}
```

В FishNet **RequireOwnership = true** автоматически отклоняет вызовы не от владельца. Ручная проверка `SenderClientId` не нужна. Чище код, меньше ошибок.

ObserversRpc и TargetRpc

③ RPC

ObserversRpc

```
// Всем наблюдателям
[ObserversRpc]
void PlayShotSoundObs
  Rpc(Vector3 pos) {
  AudioSource.PlayAt(pos);
}

// Исключая владельца
[ObserversRpc(ExcludeOwner
  = true)]
void ShowHitMarkerObs
  Rpc() { ... }
```

TargetRpc

```
// Одному клиенту
[TargetRpc]
void ShowLootDropTarget
  Rpc(
  NetworkConnection
  target,
  string itemName)
{
  ShowNotification(
    $"Вы получили {itemName}!");
}
```

NGO: **ClientRpcParams + TargetClientIds[]** → FishNet: просто параметр **NetworkConnection target**

Миграция стрельбы: построчно

③ RPC

NGO — Лекция 3, PlayerShooting.cs

```
public class PlayerShooting :  
    NetworkBehaviour {  
    void Update() {  
        if (!IsOwner) return;  
        ShootServerRpc(dir);  
    }  
  
    [ServerRpc(RequireOwnership  
        = true)]  
    void ShootServerRpc(  
        Vector3 dir,  
        ServerRpcParams p  
        = default) {  
        obj.SpawnWithOwnership(  
            OwnerClientId);  
    }  
}
```

FishNet

```
public class PlayerShooting :  
    NetworkBehaviour {  
    void Update() {  
        if (!base.IsOwner)  
            return;  
        ShootServerRpc(dir);  
    }  
  
    [ServerRpc(RequireOwnership  
        = true)]  
    void ShootServerRpc(  
        Vector3 dir) {  
        ServerManager.Spawn(obj,  
            Owner);  
    }  
}
```

Что изменилось: **IsOwner** → **base.IsOwner** | **ServerRpcParams** убран | **SpawnWithOwnership** → **ServerManager.Spawn**

④ Observers

Area of Interest — отправляем данные только
тем, кто видит объект

Проблема: все видят всё

④
Observers

✗ NGO — без AOI

Сервер синхронизирует ВСЁ со ВСЕМИ

$32 \times 1000 = 32\,000$

синхронизаций / тик

- Трафик взрывается
- Tick rate падает
- Игра лагает

✓ FishNet — с Observers

Синхронизирует только с теми, кто ВИДИТ

$32 \times \sim 50 = 1\,600$

синхронизаций / тик

- Трафик в 20× меньше
- Стабильный tick rate
- Масштабируется

Вспоминаем рычаг RELEVANCY из Лекции 2 — «кому отправляем». Observers — это реализация в FishNet.

Что такое Observer?

NetworkObject «Монстр»

- Observer: Клиент А (30м) ←  **ВИДИТ** → получает SyncVar + ObserversRpc
- Observer: Клиент В (45м) ←  **ВИДИТ** → получает SyncVar + ObserversRpc
- Клиент С (120м) ←  **НЕ ВИДИТ** → НЕ получает ничего

Observer = клиент, для которого выполнены все ObserverConditions данного объекта

Когда Клиент С подойдёт ближе → станет Observer → получит полное состояние объекта (как late-join в NGO)

Встроенные ObserverConditions

④
Observers

DistanceCondition

Расстояние видимости
MaximumDistance = 50
Основной инструмент AOI

SceneCondition

Только клиенты в той же сцене
Автоматически при сетевой загрузке сцен

OwnerOnlyCondition

Только владелец объекта
Приватные данные (инвентарь)
Экономит трафик личных данных

HostOnlyCondition

Только хост
Серверные объекты (GameManager)
Редко используется

Добавь компоненты в Inspector: **NetworkObserver** + **DistanceCondition** — и трафик уменьшится в разы. Никакого кода!

DistanceCondition: под капотом

 Объект (монстр) радиус 50м

 Клиент А (30м)  SYNC |  Клиент В (45м)  SYNC

 Клиент С (120м)  NO SYNC (за радиусом видимости)

Параметр	Значение	Описание
Maximum Distance	50 (float)	Радиус видимости в юнитах
Update Interval	1.0 (float)	Как часто проверять (секунды)

Update Interval = 1 сек означает, что Observer-проверка раз в секунду, НЕ каждый тик. Это нормально — объекты не телепортируются, секунды достаточно.

Custom ObserverCondition — свои правила

```
public class TeamCondition :  
    ObserverCondition {  
  
    public override bool ConditionMet(  
        NetworkConnection conn,  
        bool currentlyAdded,  
        out bool notProcessed)  
    {  
        notProcessed = false;  
  
        int objectTeam =  
            GetComponent<TeamComponent>().TeamId;  
        int clientTeam =  
            conn.FirstObject  
                .GetComponent<TeamComponent>().TeamId;  
  
        // Союзники видят, враги — нет  
        return objectTeam == clientTeam;  
    }  
}
```

Применения:

Fog of War

Видимость через рейкаст
RTS, MOBA

Team-based

Союзники видят, враги нет
Командные шутеры

LOD-based

Реже обновляются далёкие
ММО, открытый мир

Observers → ObserversRpc: цепная реакция

Объект «Монстр» кричит → [ObserversRpc] PlayRoarSound()

- Клиент А (Observer) →  **Получает RPC** →  Слышит рёв
- Клиент В (Observer) →  **Получает RPC** →  Слышит рёв
- Клиент С (не Observer) →  **НЕ получает** →  Тишина

 **Глобальные события** (конец раунда, чат, таймер) — отправлять через объект БЕЗ DistanceCondition!

Например: ~~GameManager без DistanceCondition → ObserversRpc доходит до всех клиентов.~~

Правило: DistanceCondition → только ближние получают ObserversRpc. БЕЗ DistanceCondition → все клиенты = аналог ClientRpc из NGO.

⑤ Миграция NGO → FishNet

Чек-лист, подводные камни, пошаговый
алгоритм

Чек-лист миграции NGO → FishNet (8 шагов)

⑤
Миграция

- ☐ Удалить пакет `com.unity.netcode.gameobjects`
- ☐ Импортировать FishNet из Asset Store / GitHub
- ☐ Заменить NetworkManager (NGO) → NetworkManager (FishNet) + добавить Tugboat
- ☐ Базовый класс: NetworkBehaviour — тот же, но из FishNet.Object
- ☐ NetworkVariable<T> → [SyncVar] атрибут
- ☐ [ClientRpc] → [ObserversRpc]
- ☐ Lifecycle: OnNetworkSpawn → OnStartClient / OnStartServer
- ☐ Spawn: `netObj.Spawn()` → `ServerManager.Spawn(obj)` или `ServerManager.Spawn(obj, owner)`

✓ **Самое важное:** Логика игры (guard-клозы, валидация, геймплей) НЕ меняется. Меняется только API-слой.

Топ-5 ошибок при переходе на FishNet

#	Ошибка	NGO код	FishNet код
1	IsOwner	IsOwner	base.IsOwner
2	OwnerClientId	OwnerClientId (ulong)	OwnerId (int) / Owner
3	Spawn	netObj.Spawn()	ServerManager.Spawn(obj)
4	Despawn	netObj.Despawn(true)	ServerManager.Despawn(obj)
5	Lifecycle	OnNetworkSpawn()	OnStartClient() + OnStartServer()

 **Совет:** Ищите ошибки компиляции — они укажут на 90% мест. Остальные 10% — runtime-проверка (особенно lifecycle).

PlayerController: до и после

5

Миграция

NGO

```
using Unity.Netcode;

public class PlayerController :
    NetworkBehaviour {
    NetworkVariable<int> health
    = new(100);

    public override void OnNetworkSpawn() {
        if (IsOwner)
            Camera.main.transform
                .SetParent(transform);
        health.OnValueChanged
            += OnHP;
    }

    void Update() {
        if (!IsOwner)
            return;
        HandleMovement();
    }
}
```

FishNet

```
using FishNet.Object;
using FishNet.Object.Synchronizing;

public class PlayerController :
    NetworkBehaviour {
    [SyncVar(OnChange = nameof(OnHP))]
    public int health = 100;

    public override void OnStartClient() {
        base.OnStartClient();
        if (base.IsOwner)
            Camera.main.transform
                .SetParent(transform);
    }
    // Отписка не нужна!

    void Update() {
        if (!base.IsOwner)
            return;
        HandleMovement();
    }
}
```

Добавляем Observers: 3 шага

⑤
Миграция

Шаг 1: Добавить NetworkObserver на каждый сетевой объект

Шаг 2: Добавить DistanceCondition

→ MaximumDistance подбираем под жанр:

FPS: 100–150 м TPS: 80–120 м MMO: 50–80 м Racing: 200–300 м

Шаг 3: Проверить, что глобальные объекты

(GameManager, ChatManager) НЕ имеют DistanceCondition

✓ Observers — ключевая причина перехода на FishNet: без правок кода можнократно сократить трафик через настройки в Inspector.

А что с задержками?

© Мост

Network Simulation: пинг 200 мс

БЕЗ Prediction:

Нажатие W → 200 мс ожидания → движение

→ Персонаж реагирует с задержкой

→ Управление становится «ватным»

→ Для шутера это неиграбельно

Включите FishNet Network Simulation (NetworkManager → Latency: 200ms) и попробуйте управление сами.

Лекция 6: Prediction, Reconciliation, Lag Compensation — как добиться мгновенного отклика даже при высоком пинге.

Что мы узнали

FishNet

- Мотивация: производительность, Observer-система, встроенный Prediction
- Архитектура: NetworkManager (не синглтон), ServerManager/ClientManager, TransportManager (Tugboat/Bayou), TimeManager (тики)
- SyncVar / SyncType: [SyncVar], SyncList, SyncDictionary, SyncHashSet
- RPC: [ServerRpc], [ObserversRpc], [TargetRpc]
- Observers (AOI): DistanceCondition, SceneCondition, Custom Conditions, влияние на ObserversRpc